CNSS Practical Examination Solutions

All cryptographic and framing practical programs using 0 to 25 indexing (a=0, ..., z=25).

1. Additive Cipher

```
def additive_cipher(text, key):
    text = text.replace(' ', '').lower()
    cipher_text = ''
    for char in text:
        if char.isalpha():
            p = ord(char) - ord('a')
            c = (p + key) % 26
            cipher_text += chr(c + ord('a'))
    return cipher_text

# Usage:
print(additive_cipher('this message is easy to encrypt but hard to find the
key', 15))
```

2. Caesar Cipher

```
def caesar_cipher(text):
    text = text.replace(' ', '').lower()
    key = 3
    cipher_text = ''
    for char in text:
        if char.isalpha():
            p = ord(char) - ord('a')
            c = (p + key) % 26
            cipher_text += chr(c + ord('a'))
    return cipher_text

# Usage:
print(caesar_cipher('this message is easy to encrypt but hard to find the
key'))
```

3. Shift Cipher

```
def shift_cipher(text, key):
    text = text.replace(' ', '').lower()
    cipher_text = ''
    for char in text:
        if char.isalpha():
            p = ord(char) - ord('a')
```

```
            c = (p + key) % 26
            cipher_text += chr(c + ord('a'))
    return cipher_text

# Usage:
print(shift_cipher('this message is easy to encrypt but hard to find the key',
15))
```

4. Affine Cipher

```
def affine_cipher(text, a, b):
    text = text.replace(' ', '').lower()
    cipher_text = ''
    for char in text:
        if char.isalpha():
            p = ord(char) - ord('a')
            c = (a * p + b) % 26
            cipher_text += chr(c + ord('a'))
    return cipher_text

# Usage:
print(affine_cipher('hello', 7, 2))
```

5. Autokey Cipher

```
def autokey_cipher(text, initial_key):
    text = text.replace(' ', '').lower()
    key_stream = [initial_key] + [ord(c) - ord('a') for c in text[:-1]]
    cipher_text = ''
    for i, char in enumerate(text):
        p = ord(char) - ord('a')
        k = key_stream[i]
        c = (p + k) % 26
        cipher_text += chr(c + ord('a'))
    return cipher_text

# Usage:
print(autokey_cipher('the house is being sold tonight', 7))
```

6. Playfair Cipher

```
def playfair_cipher(text, secret_key):
    text = text.replace(' ', '').lower().replace('j', 'i')
    secret_key = secret_key.replace(' ', '').lower().replace('j', 'i')

    matrix_chars = []
    for char in secret_key + 'abcdefghiklmnopqrstuvwxyz':
```

```
        if char not in matrix_chars:
            matrix_chars.append(char)
    matrix = [matrix_chars[i:i+5] for i in range(0, 25, 5)]

    digraphs = []
    i = 0
    while i < len(text):
        a = text[i]
        b = text[i+1] if i+1 < len(text) else 'x'
        if a == b:
            digraphs.append((a, 'x'))
            i += 1
        else:
            digraphs.append((a, b))
            i += 2

    def find_pos(char):
        for r, row in enumerate(matrix):
            if char in row:
                return r, row.index(char)
        return None

    cipher = ''
    for a, b in digraphs:
        r1, c1 = find_pos(a)
        r2, c2 = find_pos(b)
        if r1 == r2:
            cipher += matrix[r1][(c1 + 1) % 5] + matrix[r2][(c2 + 1) % 5]
        elif c1 == c2:
            cipher += matrix[(r1 + 1) % 5][c1] + matrix[(r2 + 1) % 5][c2]
        else:
            cipher += matrix[r1][c2] + matrix[r2][c1]
    return cipher

# Usage:
print(playfair_cipher('the key is hidden under the door pad', 'guidance'))
```

7. Vigenère Cipher

```
def vigenere_cipher(text, keyword):
    text = text.replace(' ', '').lower()
    keyword = keyword.replace(' ', '').lower()
    key_extended = keyword * (len(text) // len(keyword)) + keyword[:len(text)
% len(keyword)]
    cipher_text = ''
    for p_char, k_char in zip(text, key_extended):
        p = ord(p_char) - ord('a')
        k = ord(k_char) - ord('a')
```

```
        c = (p + k) % 26
        cipher_text += chr(c + ord('a'))
    return cipher_text

# Usage:
print(vigenere_cipher('she is listening', 'pascal'))
```

8. Hill Cipher

```
import numpy as np

def hill_cipher(text):
    text = text.replace(' ', '').lower()
    if len(text) % 2 != 0:
        text += 'x'
    key_matrix = np.array([[3, 2], [5, 7]])

    cipher_text = ''
    for i in range(0, len(text), 2):
        vector = np.array([[ord(text[i]) - ord('a')], [ord(text[i+1]) -
ord('a')]])
        result = np.dot(key_matrix, vector) % 26
        cipher_text += chr(result[0, 0] + ord('a')) + chr(result[1, 0] +
ord('a'))
    return cipher_text

# Usage:
print(hill_cipher('we live in an insecure world'))
```

9. Character Count Framing Technique

```
def character_count_framing(frames):
    transmitted_stream = ''
    for frame in frames:
        count = len(frame) + 1
        transmitted_stream += str(count) + frame
    return transmitted_stream

# Usage:
print(character_count_framing(['AB', 'ESC', 'FLAG']))
```

10. Byte Stuffing (Character Stuffing)

```
def byte_stuffing(frame, flag='FLAG', esc='ESC'):
    stuffed = []
    for word in frame:
        if word == flag or word == esc:
```

```
            stuffed.append(esc)
        stuffed.append(word)
    return [flag] + stuffed + [flag]

# Usage:
print(byte_stuffing(['A', 'B', 'ESC', 'FLAG']))
```

11. Bit Stuffing

```
def bit_stuffing(data_bits):
    stuffed = ''
    count = 0
    for bit in data_bits:
        if bit == '1':
            count += 1
            stuffed += bit
            if count == 5:
                stuffed += '0'
                count = 0
        else:
            count = 0
            stuffed += bit
    return '01111110' + stuffed + '01111110'

# Usage:
print(bit_stuffing('010001111111011'))
```